

Egyptian Currency Detection — Production Report (v2.0)

Date: 2026-06-18 **Author:** MazenBasha (graduation project) **Repo path:**

D:\roshdi-grad-project\Egyptian Currency Detection **Scope:** This report documents the v2.0 production-hardening pass. It supplements `PROJECT_REPORT.md` (2026-05-02) and `UPDATE_REPORT.md` (2026-05-21), which describe v1 architecture and the multi-note counting fix respectively.

1. Executive summary

The detector was already functional and well-documented but lacked the boring-but-critical machinery that lets software run in front of real users: input validation, structured logs, confidence-aware outputs, automated tests, performance benchmarks, and a way to track which model is in production.

This pass adds all of that **without changing the architecture** — no API server, no async queue, no database, no extra dependencies beyond `pytest` and `psutil`. The system is still a single CLI script over a single YOLOv8-nano model. Every old command still works.

Dimension	Before (v1.x)	After (v2.0)
Crash on bad input?	Yes — <code>FileNotFoundError</code> , <code>ValueError</code>	No — returns structured error JSON
Audit trail	<code>print()</code> to console only	JSONL per run in <code>outputs/logs/</code>
Confidence transparency	Raw float per box	Risk band + class-confusion warning
Automated tests	None	37 tests in <code>pytest</code> , 5.5s run
Performance data	Anecdotal	Repeatable benchmark (29 img/s on CPU)
Model versioning	Hard-coded path in <code>config.py</code>	JSON registry, switch with one call
JSON schema	3 keys	Schema v2 with metadata + summary, old keys preserved

Production-readiness scorecard moved from ~54% to ~75% (per the criteria laid out in the v2.0 requirements doc). The 25% gap remaining is mostly deployment concerns (CI/CD, monitoring, signed releases) that the spec explicitly excluded.

2. Project background (one-page recap)

The full history lives in the earlier reports; this is just enough to make the rest of this document standalone.

Goal. Given an image, video frame, or webcam feed, detect every visible Egyptian banknote, classify each as one of seven denominations (1, 5, 10, 20, 50, 100, 200 EGP), count per class, and report the total EGP in view.

Model. YOLOv8-nano (Ultralytics), fine-tuned on a labelled multi-note dataset. Trained run lives at `outputs/runs/train_finetune/`. The model file is ~6 MB — small enough to run on CPU at ~30 img/s and on mobile via the exported TorchScript-Lite and TFLite variants.

Dataset. `data_yolo/` (real-photo training + held-out test split) and `data_synth/` (synthetic multi-note compositions used for the counting fix). ~290 images in the test split.

Inference entry point. `python src/detect.py` — single CLI accepting `--image`, `--image-dir`, `--video`, or `--webcam`. Every invocation writes an annotated media file plus a JSON record.

3. What v2.0 added — module by module

Eight focused improvements, in priority order.

3.1 Input validation (`src/validators.py`)

A defensive layer that runs before every inference call. Catches the failure modes that used to crash the CLI:

- File missing, path is a directory, extension unsupported, file unreadable.
- Image corrupted (header doesn't decode).
- Image smaller than 100 pixels total (no real note will ever fit).
- Image larger than 50 million pixels (RAM blow-up on 200 MP phone shots).

API. `InputValidator` static methods, all returning tuples so callers never have to wrap a try/except:

Method	Returns
<code>validate_image_path(path)</code>	<code>(bool, error_msg)</code>
<code>validate_image_content(path)</code>	<code>(bool, error_msg, {width, height, pixels, aspect_ratio, mode})</code>
<code>validate_folder(path)</code>	<code>(bool, error_msg, [image_paths])</code>

`build_error_json(error_msg, image_path)` packages a failed validation into the canonical error envelope:

```
{ "success": false, "error": "...", "image": "...",  
  "detections": [], "counts": {}, "total": 0 }
```

That envelope is written to `outputs/detections/<stem>.json` instead of raising — downstream callers always read a valid JSON file.

3.2 Structured logging (`src/logging_config.py`)

One logger, two handlers:

- **File handler** — JSONL to `outputs/logs/detection_<UTC-timestamp>.jsonl`,

one self-contained event per line. Parseable by `jq` or any log shipper.

- **Console handler** — human-readable line for live monitoring.

Every record carries `timestamp`, `level`, `message`, `module`, `line_number`. Optional fields injected via `extra={}` from `detect.py`: `image_path`, `num_detections`, `inference_ms`, `confidence_min`, `confidence_max`, `error_type`, `model_version`.

Example trace from a real run:

```
{"timestamp": "2026-06-18T...", "level": "INFO", "message": "Starting detection",  
  "module": "detect", "line_number": 99, "image_path": "data_yolo/.../1_1.100.jpg"}  
{"timestamp": "2026-06-18T...", "level": "INFO", "message": "Detection complete",  
  "module": "detect", "line_number": 155, "image_path": "...", "num_detections": 1,  
  "inference_ms": 298.308, "confidence_min": 0.842, "confidence_max": 0.842}
```

Errors emit a record with `error_type` (`validation`, `decode`, `inference`) so the operator can categorize failures from `jq` without reading the message string.

3.3 Confidence & risk flags (`src/confidence_metrics.py`)

Raw YOLO confidence is a single float that hides two problems:

- *How safe is this number?* — A 0.62 box and a 0.95 box both pass the

default NMS but have very different reliability.

- *Is this class historically confused?* — Our model regularly mistakes 50

EGP for 100 EGP (almost identical orange-ish color); 1 EGP has fewer training examples than any other class.

`ConfidenceAnalyzer.add_uncertainty_flags()` attaches two fields per box:

risk band	Threshold	Meaning
low	$\text{conf} \geq 0.90$	Trust it
medium	$0.75 \leq \text{conf} < 0.90$	Borderline — review on high-stakes use
high	$0.50 \leq \text{conf} < 0.75$	Likely misread
reject	$\text{conf} < 0.50$	Don't act on this

Class	Warning fires when	Message
100_EGP	$\text{conf} < 0.95$	"May be confused with 50_EGP (similar color)"
50_EGP	$\text{conf} < 0.95$	"May be confused with 100_EGP (similar color)"
1_EGP	$\text{conf} < 0.90$	"Rare class, fewer training examples"

`ConfidenceAnalyzer.confidence_report()` summarises a batch with `min`, `max`, `mean`, `std`, per-band counts, and a list of risky detections suitable for human review.

`ConfidenceAnalyzer.filter_by_confidence(dets, threshold)` is the simple helper for callers that just want a hard cutoff.

3.4 Enhanced JSON schema v2 (`src/utils.py`)

Schema bumped from "three fields" to "rich, self-describing record" while keeping the old fields in place so existing parsers don't break.

```
{
  "schema_version": "2.0",
  "metadata": {
    "timestamp": "2026-06-18T00:49:05.627",
    "image": "data_yolo/.../1_1.100.jpg",
    "model_version": "outputs/runs/train_finetune/weights/best.pt",
    "inference_time_ms": 298.308,
    "image_size": [201, 100]
  },
  "detections": [
    { "id": 0, "class": "1_EGP", "confidence": 0.842,
      "risk": "medium", "warning": "Rare class, fewer training examples",
      "bbox": [0.058, 0.0, 125.58, 99.98],
```

```

    "bbox_normalized": [0.0003, 0.0, 0.6248, 0.9998],
    "area_pixels": 12549 }
  ],
  "summary": {
    "num_detections": 1, "counts": {"1_EGP": 1}, "total_egg": 1,
    "confidence": { "min": 0.842, "max": 0.842, "mean": 0.842, "std": 0.0 },
    "high_risk_count": 0
  },
  "counts": {"1_EGP": 1},
  "total": 1,
  "success": true,
  "confidence_report": { ... }
}

```

Why two coordinate systems? `bbox` is pixel-space (what the model sees, what the annotated overlay uses). `bbox_normalized` is 0–1 of image size, which is the format used by mobile renderers and the YOLO label format — this saves the consumer one division.

`schema_version` is the safety net. A future v3 schema can add fields freely; parsers can switch on the version field.

3.5 Test suite (tests/)

Run with `pytest tests/ -v --tb=short`. Three files, 37 tests total, every test passes against Python 3.11 + PyTorch CPU.

File	What it covers
<code>test_validators.py</code>	Missing files, unsupported extensions, corrupt JPEGs, oversized / undersized images, empty / valid folders, all three supported extensions parametrized
<code>test_utils.py</code>	<code>summarize()</code> (empty / single / mixed / unknown class), v2 schema fields, normalized bbox math, confidence stats, all four risk band thresholds, warning rules
<code>test_adversarial.py</code>	All-white image, all-black, gaussian noise, motion blur, 90° rotation, tiny note in large canvas, unusual aspect ratio — model must not crash on any of them

All fixtures are built in-memory with PIL/numpy/tempfile. No external test images required. `conf_test.py` patches `sys.path` so tests can import `validators`, `utils`, etc. without an editable install.

The adversarial suite reuses the real trained model — it loads the active weights from `config.DEFAULT_WEIGHTS` and `pytest.skip()`s if they're unavailable, so the suite still runs cleanly on a fresh clone with no weights file.

3.6 Performance profiling (src/profiling.py)

Two static methods on `PerformanceProfiler`:

- `profile_inference(func, *args, **kwargs)` — wraps a single call.

Returns `elapsed_ms`, `memory_used_mb`, `memory_peak_mb`, `cpu_percent`, `result`, `error`. Uses `time.perf_counter` for wall time, `tracemalloc` for Python-side peak, `psutil.Process()` for OS-level RSS.

- `profile_batch(model, image_paths, batch_size=4)` — drives the model

end-to-end across a list of images. Returns `num_images`, `batch_size`, `total_time_ms`, `avg_time_per_image_ms`, `avg_memory_per_image_mb`, `peak_memory_mb`, `images_per_second`, `times_per_batch`.

The first batch is always slower (model warm-up, dispatch caches); subsequent batches stabilize at the true steady-state cost. A representative run on a 20-image subset, batch size 4, CPU-only:

Metric	Value
Total time	686.9 ms
Mean latency	34.3 ms/image
Throughput	29.1 images/sec
Peak RSS	375.6 MB
Per-batch times (ms)	277, 106, 100, 99, 95

This is more than fast enough for the target use case (still-image counting from a phone snapshot) and roughly real-time on a laptop CPU.

3.7 Model registry (`src/model_registry.py`)

A flat JSON list at `outputs/model_registry.json` — one entry per trained model. Entries are mutable in place (re-registering the same path updates metrics rather than appending). Exactly one entry is `active: true` at a time.

`ModelRegistry` static methods:

Method	Effect
<code>register_model(path, metrics, notes)</code>	Append (or update) an entry. First registered model auto-activates.
<code>set_active_model(index)</code>	Switch the active flag.
<code>get_active_model()</code>	Return the active weights path (or <code>None</code>).
<code>list_models()</code>	Print a formatted table; return the registry list.
<code>load_registry() / save_registry()</code>	Raw read/write helpers.

Sample registry after registering two models:

```
#  ACTIVE  mAP50  mAP50-95  SIZE_MB  PATH
0   yes    0.995   0.989     5.929   outputs/runs/train_finetune/weights/best.pt
    notes: Original fine-tune
1          0.982   0.971     5.919   outputs/runs/train2/weights/best.pt
    notes: Earlier baseline
```

`src/config.py` was updated to read `DEFAULT_WEIGHTS` from the registry, with a fallback to the legacy hard-coded path so the project still works on a fresh clone (where no registry file exists yet):

```
def _resolve_default_weights() -> str:
    try:
        from model_registry import ModelRegistry
        active = ModelRegistry.get_active_model()
        if active and os.path.exists(active):
            return active
    except Exception:
        pass
    return _FALLBACK_WEIGHTS
```

```
DEFAULT_WEIGHTS = _resolve_default_weights()
```

3.8 Integration in `src/detect.py`

The new modules are wired into the existing `run_on_image` pipeline:

- `setup_logging()` at module import — produces a single `logger`.
- `InputValidator.validate_image_content(path)` — bail with error JSON if

the image is bad. No exception escapes the function.

- `model.predict(...)` wrapped in try/except — inference exceptions are

logged and converted to error JSON.

- After NMS: `ConfidenceAnalyzer.add_uncertainty_flags(detections)` and

`ConfidenceAnalyzer.confidence_report(detections)`.

- ``build_output_dict(detections, image_path, inference_ms, model_version,`

`image_size, confidence_report)`` produces the v2 schema.

- `START / COMPLETE / ERROR` log lines with the relevant `extra={}` fields.

`run_on_folder` was updated to use `InputValidator.validate_folder` for the listing step (replacing the old ad-hoc `os.listdir + extension` filter), keeping the per-image path identical.

Stream paths (`--video`, `--webcam`) were left untouched — they don't operate on disk inputs, so the validator doesn't apply, and per-frame JSONL logging on a 25 fps stream would drown the log directory.

4. Updated file layout

Egyptian Currency Detection/

```
■■■ src/
■   ■■■ config.py           # updated: DEFAULT_WEIGHTS reads from registry
■   ■■■ train.py           unchanged
■   ■■■ detect.py          # updated: validators + logging + confidence
■   ■■■ evaluate.py        unchanged
■   ■■■ utils.py           # updated: schema v2
■   ■■■ bootstrap_yolo_dataset.py unchanged
■   ■■■ validators.py      ★ NEW
■   ■■■ logging_config.py  ★ NEW
■   ■■■ confidence_metrics.py ★ NEW
■   ■■■ profiling.py       ★ NEW
■   ■■■ model_registry.py  ★ NEW
■
■■■ tests/                  ★ NEW
■   ■■■ __init__.py
■   ■■■ conftest.py        (sys.path bootstrap for the suite)
■   ■■■ test_validators.py
■   ■■■ test_utils.py
■   ■■■ test_adversarial.py
■
■■■ data_yolo/              real-photo dataset
■■■ data_synth/            synthetic multi-note dataset
■■■ legacy/                previous single-image classifier project
■
■■■ outputs/
■   ■■■ runs/              trained-model directories
■   ■■■ detections/        annotated jpg + json per inference
■   ■■■ logs/              ★ JSONL audit trail, auto-created
■   ■■■ model_registry.json ★ active model + history
■   ■■■ best_model.pth      legacy classifier checkpoint
```

■	
■■■ requirements.txt	# +pytest, +psutil
■■■ README.md	# +"New Features (v2.0)" section
■■■ PROJECT_REPORT.md	v1 design report (2026-05-02)
■■■ UPDATE_REPORT.md	v1.5 multi-note counting fix (2026-05-21)
■■■ MODEL_DESCRIPTION.md	model card
■■■ FILE_GUIDE.md	file-by-file documentation
■■■ PRODUCTION_REPORT.md	★ THIS FILE

5. How to use the project today

5.1 Install

```
pip install -r requirements.txt
```

Now pulls in `pytest>=7.0` and `psutil>=5.9` in addition to the original ultralytics / torch / opencv / pillow stack.

5.2 Run inference (single image)

```
python src/detect.py --image data_yolo/images/test/1__1.100.jpg --no-show
```

Produces:

- `outputs/detections/1__1.100_annotated.jpg`
- `outputs/detections/1__1.100.json` (schema v2)
- A line in `outputs/logs/detection_<timestamp>.jsonl`

5.3 Batch over a folder

```
python src/detect.py --image-dir path/to/images --no-show
```

Same per-image artifacts, all under `outputs/detections/<folder-name>/`.

5.4 Video / webcam

```
python src/detect.py --video clip.mp4
python src/detect.py --webcam --camera 0
```

These still work exactly like v1.

5.5 List / switch models

```
python -c "from src.model_registry import ModelRegistry; ModelRegistry.list_models()"
python -c "from src.model_registry import ModelRegistry; ModelRegistry.set_active_model(0)"
```

After switching, the next `detect.py` run uses the new weights without editing `config.py`.

5.6 Run tests

```
pytest tests/ -v
```

Expected output: `37 passed in ~6s`.

5.7 Benchmark performance

```
from src.profilig import PerformanceProfiler
from src.utils import load_model
from src.model_registry import ModelRegistry
import glob

model = load_model(ModelRegistry.get_active_model())
```

```
images = glob.glob("data_yolo/images/test/*.jpg")[:50]
print(PerformanceProfiler.profile_batch(model, images, batch_size=4))
```

5.8 Tail the audit log

```
Get-Content outputs/logs/detection_*.jsonl -wait
```

Or with `jq` for filtered queries:

```
jq 'select(.level=="ERROR")' outputs/logs/detection_*.jsonl
jq 'select(.num_detections>0) | {image_path, num_detections, inference_ms}' outputs/logs/*.jsonl
```

6. Behaviour changes worth knowing

These are the user-visible differences when upgrading a v1 deployment to v2.0:

- **Bad-input behaviour.** Code that previously caught

`FileNotFoundError` from `detect.py` will no longer see that exception — the script now writes an error JSON and returns normally. Update downstream pipelines to read `success: false` from the JSON instead of relying on the process exit code.

- **JSON output is larger.** The old three-key JSON is still present as

top-level fields, but new fields (`metadata`, `summary`, `detections[*].id`, `risk`, `bbox_normalized`, etc.) make the file ~3x larger. Old parsers keep working; new parsers should switch on `schema_version`.

- **New writes to `outputs/logs/` on every run.** One file per process

invocation. They are small (kilobytes) but accumulate. Rotation / cleanup is the operator's responsibility — the project itself does not delete logs.

- **`config.DEFAULT_WEIGHTS` is now dynamic.** It reads from

`outputs/model_registry.json` at import time. If you previously edited `config.py` to change the model, prefer `ModelRegistry.set_active_model(...)` instead — `config.py` is now declarative.

No change to the model itself, the training pipeline, the evaluation metrics, or the inference output (boxes and confidences are byte-for-byte identical to v1.5).

7. Test results

```
$ pytest tests/ -v --tb=short
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.1.0
collected 37 items

tests/test_adversarial.py ..... PASSED (7 tests)
tests/test_utils.py ..... PASSED (15 tests)
tests/test_validators.py ..... PASSED (15 tests)

===== 37 passed in 5.52s =====
```

Breakdown:

- **Validators (15 tests):** path checks, content checks, corrupt JPEG,

oversized image (8000×7000), undersized image (5×5), parametrized format support (.jpg, .png, .bmp), folder validation incl. empty folder and non-image file filtering.

- **Utils + confidence (15 tests):** `summarize()` for empty / single /

multi-class / unknown class, schema v2 required fields, metadata fields, normalized bbox arithmetic, confidence statistics, all four risk band thresholds, 50/100 EGP warning, threshold suppression, empty report.

- **Adversarial (7 tests):** all-white, all-black, gaussian noise, motion

blur, 90° rotation, scaled-down note in large canvas, unusual aspect ratio. All run the real model.

8. Constraints honoured

The v2.0 brief was explicit about what *not* to do. Compliance:

Constraint	Status
No API servers (FastAPI / Flask / Django)	✓ none added
No architectural changes	✓ same single YOLOv8 model, no DB
No async / queues	✓ inference is synchronous
Monolithic — single CLI entry point	✓ <code>python src/detect.py</code>
Backward compatible — old commands still work	✓ all v1 flags accepted
Old JSON parseable	✓ legacy keys preserved
Total time budget: 10 working hours	✓

The only files in `outputs/` that were not present in v1 are `outputs/logs/*` and `outputs/model_registry.json`, both of which are auto-created on demand and don't affect anything else.

9. Acceptance criteria — final checklist

Category	Requirement	Status
Code quality	No uncaught exceptions	✓ all error paths return JSON
	All imports resolvable	✓ verified
	requirements.txt updated	✓ +pytest, +psutil
	Existing project style	✓
Testing	pytest tests/ -v passes 100%	✓ 37/37
	Unit + adversarial coverage	✓
	Python 3.11 + PyTorch CPU	✓
Functionality	Validation prevents crashes	✓
	JSONL logging to outputs/logs/	✓
	Confidence metrics in output JSON	✓
	Model registry list/switch/register	✓
	Profiling measures inference time	✓ 29 img/s
Backwards compat.	Old CLI still works	✓
	Old JSON keys preserved	✓
	No breaking changes to config/train/evaluate	✓ train/evaluate untouched
	Existing weights still loadable	✓
Documentation	README updated with new features	✓
	Examples for each feature	✓
	All new modules carry docstrings	✓

10. What was deliberately *not* done

Out of scope for v2.0, mentioned here so they aren't mistaken for oversights:

- **CI pipeline.** No GitHub Actions or pre-commit hooks. Tests are manual-runnable.
- **Per-frame logging on video streams.** A 25 fps webcam would generate ~90,000 log lines per hour. Stream paths intentionally bypass the per-event logger.
- **Detection deduplication across video frames.** The total EGP value on a webcam feed still re-counts the same note every frame; that's a product decision, not a robustness one.
- **Auth on the registry.** Anyone with write access to `outputs/` can change the active model. This is a single-developer grad project; the registry is a notebook, not a security boundary.
- **GPU benchmarks.** The dev machine is CPU-only. The profiler will run

identically on GPU once a CUDA torch wheel is installed.

- **Schema v3.** v2 is the right shape for the current consumers. The version field exists so a future schema can be added without breaking anyone.

11. Where to look next

When the project picks up again, the highest-leverage follow-ups are:

- **Add a CI workflow** that runs `pytest tests/ -v` on every push. The test suite is small and self-contained; this is half an hour of work and pays back forever.
- **Auto-rotate** `outputs/logs/`. Either by date or by file size. Easy addition to `logging_config.setup_logging` using `logging.handlers.TimedRotatingFileHandler`.
- **Confidence calibration.** Currently the risk bands are uniform across classes. If the test split shows calibration errors per class, the bands could be learned rather than hard-coded.
- **Mobile integration.** The TorchScript-Lite and TFLite artifacts under `outputs/runs/train_finetune/weights/` are ready to go; only the Flutter shell is missing.

12. References

- `PROJECT_REPORT.md` — v1 design and rationale (2026-05-02)
- `UPDATE_REPORT.md` — multi-note counting fix (2026-05-21)
- `MODEL_DESCRIPTION.md` — model card, classes, hyperparameters
- `FILE_GUIDE.md` — file-by-file walkthrough
- `README.md` — install + quick-start